

Mazeed+

Full-Stack E-Commerce Platform

Project Documentation

ITI Summer Training 2026 — Full Stack .NET Track

ASP.NET Core MVC | Entity Framework Core 8 | SQL Server

Live Demo: <https://mazeed.runasp.net/>

*Document Version 1.0
September 2026*

1. Introduction

1.1 Project Overview

Mazeed+ is an ASP.NET Core MVC e-commerce web application built with .NET 8 as part of the ITI Summer Training 2026 Full Stack .NET track. The application implements a complete online retail experience: shoppers can browse a categorized catalog of branded items, filter and search products, select item variants (size, color, SKU-level stock), maintain a cart and wishlist, complete checkout with an integrated payment provider, leave reviews, and receive real-time notifications. Administrators manage the catalog, orders, discount rules, users, and roles through a dedicated back office and review sales analytics on a dashboard.

The system is organized as a three-project layered solution — a presentation layer (Mazeed.PL), a business logic layer (Mazeed.BLL), and a data access layer (Mazeed.DAL) — backed by a SQL Server database managed through Entity Framework Core 8 with a code-first modeling approach.

1.2 Project Objectives

- Deliver a functionally complete online storefront covering discovery, purchasing, and post-purchase engagement (reviews, notifications, order history).
- Demonstrate a clean, layered ASP.NET Core MVC architecture with a clear separation between HTTP concerns, business rules, and persistence.
- Apply ASP.NET Core Identity for authentication and role-based authorization, including third-party sign-in through Google OAuth.
- Integrate a real payment provider (Paymob) for checkout, and a real-time communication channel (SignalR) for notifications.
- Provide an administrative back office for catalog, order, discount, and user/role management, supported by dashboard analytics.
- Establish and follow consistent coding, database, and documentation standards suitable for a team-based training project.

1.3 Project Scope

The current scope of Mazeed+ covers the following capabilities, as reflected in the implemented controllers and services:

- Public catalog browsing, category-scoped browsing, search with autocomplete suggestions, filtering by brand/category/price, and sorting, including a recommendation-based default ordering.
- Item detail pages showing variants, available colors and sizes, and related or frequently-bought-together items.
- Shopper account lifecycle: registration with email confirmation, cookie-based login, password reset, Google external login, and profile management including a profile photo.
- Cart and wishlist management, checkout with shipping details, discount/promotion code application, and Paymob-based payment initiation and confirmation.
- Item reviews and rating summaries, both for individual items and in batch for listing pages.
- User notifications delivered and read through a SignalR-backed notification center, including administrator broadcasts.

- Administrative management of brands, categories, items, item variants, discount rules, orders (status updates), users, and roles, plus a sales dashboard with filterable KPIs and charts.

Out of scope for the current version: multi-vendor/marketplace selling, native mobile applications, multi-currency or multi-language storefronts, and automated test coverage (the solution does not currently include a test project).

1.4 Target Users

User group	Description
Guest / anonymous visitor	Browses the public catalog, item details, and static pages (About, Help, Terms, Contact) without an account.
Registered shopper	Authenticated customer who manages a cart and wishlist, checks out, pays via Paymob, tracks orders, writes reviews, and receives notifications.
Administrator	Authenticated user in the Admin role who manages catalog data, discount rules, orders, users, roles, and views dashboard analytics.
Training reviewer / instructor	Evaluates the project as an ITI Summer Training deliverable against functional and architectural expectations.

1.5 Business Context

Mazeed+ was developed as a capstone-style training project to demonstrate practical, production-style skills in the ASP.NET Core / EF Core / SQL Server stack. While it is an educational deliverable rather than a commercial deployment, it is built to reflect real e-commerce concerns: catalog and inventory modeling, secure authentication, payment processing, order lifecycle management, and administrative operations — the same concerns a small-to-medium online retailer would need addressed in production software.

2. System Overview

2.1 System Description

Mazeed+ is a server-rendered ASP.NET Core MVC application, using Razor views for the storefront and administrative screens, combined with a set of API-oriented controllers under the /api prefix for cart, wishlist, reviews, discounts, and notifications. The application is a modular monolith: a single deployable web application internally separated into presentation, business logic, and data access layers, all communicating with one SQL Server database through Entity Framework Core.

2.2 Key Features

- Catalog management for brands, categories, items, variants, and photos.
- Search, filtering, recommendations, reviews, and rating summaries.
- Shopping cart, wishlist, shipping details, checkout, order history, and Paymob payments.
- Discount rules and promotion codes, including spend-threshold automatic discounts.
- ASP.NET Core Identity registration, email workflows, password reset, and Google sign-in.
- Admin management for users, roles, catalog, orders, discounts, and dashboard analytics.
- Real-time notifications delivered through a SignalR hub, with administrator broadcast capability.
- API-oriented endpoints under /api, including cart, discounts, notifications, reviews, and wishlist.

2.3 User Roles & Permissions

Role	Typical permissions
Public / anonymous	View catalog, item details, category pages, public reviews and rating summaries, static pages; register or log in.
Authenticated shopper	All public permissions plus cart, wishlist, checkout/payment, order history, profile management, creating/editing/deleting own reviews, and viewing/managing own notifications.
Administrator (Admin role)	All shopper permissions plus catalog CRUD (brands, categories, items, variants), discount rule management, order status updates, user and role management, dashboard analytics, and notification broadcasts.

Authorization is enforced with ASP.NET Core's [Authorize] attribute, applied at the class level for admin-only controllers (e.g., BrandController, CategoriesController, ItemsController, ItemVariantsController, AdminController) and at the action level where a controller mixes public and authenticated behavior (e.g., ReviewController, DiscountController, NotificationController).

2.4 System Workflow

Most browser requests follow a consistent path through the layered architecture:

- An MVC controller in Mazeed.PL receives the request and validates route, form, or API input.
- The controller calls a service abstraction from Mazeed.BLL/Services/Abstraction.
- The service applies business rules, maps entities and view models through AutoMapper where appropriate, and uses a repository or IUnitOfWork.
- Mazeed.DAL queries AppDbContext through Entity Framework Core and returns the result to the service.
- The controller returns a Razor view, redirect, or API response.

Notifications are pushed through NotificationHub and SignalRNotificationPusher. External integrations are isolated in the BLL service layer: email uses configured SMTP settings, and payment operations use Paymob.

2.5 Technology Stack

Area	Project technology
Web	ASP.NET Core MVC, Razor Views, .NET 8
Business logic	C#, dependency injection, AutoMapper, MemoryCache
Data access	Entity Framework Core 8, SQL Server, repositories, unit of work
Authentication	ASP.NET Core Identity with long keys and Google OAuth
Payments	Paymob
Real-time updates	SignalR notifications
Email	SMTP through EmailSettings
Documentation	Draw.io ERD in docs/ERD.drawio

3. Functional Requirements

3.1 Functional Requirements Overview

The functional requirements below are organized by capability area and are traceable to the controllers, services, and routes implemented in Mazeed.PL. Requirements are grouped into user management, authentication and authorization, core storefront features, administrative features, and notifications/reporting.

3.2 User Management

- A visitor shall be able to register a new account, providing profile details, and receive an email confirmation workflow before or alongside first sign-in.
- A registered user shall be able to view and edit their profile, including first/last name, birth date, gender, address (street/city/governorate), phone number, and a profile photo (uploaded and validated by `AuthController.UpdateProfileImage`).
- A registered user shall be able to view their order history from the profile page.
- An administrator shall be able to list, create, edit, and delete application users, and manage their role assignments (`AdminController.Users`, `CreateUser/EditUser/DeleteUser`, `ManageUserRoles`).
- An administrator shall be able to create and delete roles (`AdminController.CreateRole / DeleteRole`).

3.3 Authentication & Authorization

- The system shall authenticate users using ASP.NET Core Identity with cookie-based sign-in, a login path of `/Auth/Login`, a logout path of `/Auth/Logout`, and an access-denied path of `/Error/403`.
- The system shall support registration, login, forgot/reset password, and email confirmation flows through `AuthController`, coordinating `IAuthService`, `IUserService`, and `IEmailService`.
- The system shall support Google external sign-in, including first-time external account confirmation (`ExternalLogin`, `ExternalLoginCallback`, `ExternalLoginConfirmation`).
- Already-authenticated users shall be redirected away from anonymous-only authentication actions, except for profile, logout, city lookup, and profile-image actions.
- Administrative controllers and actions shall require the Admin role via `[Authorize(Roles = "Admin")]`.
- Unauthenticated requests to API endpoints shall receive HTTP 401 rather than an HTML login redirect.

3.4 Core System Features

Catalog Discovery

- The system shall present a browsable catalog (`HomeController.Index` / `Index_Filter`, `StoreController.Index`) with filtering by category, brand, price range, color, and size, and sorting including a recommendation-based default.
- The system shall provide category-scoped browsing (`HomeController.Category`) and item detail pages with variant, color/size, and related-item information (`HomeController.Details`).
- The system shall provide search autocomplete returning up to eight distinct item, brand, or category name suggestions (`StoreController.GetSuggestions`).

Cart, Wishlist, and Checkout

- An authenticated user shall be able to view, add to, update the quantity of, remove from, and clear their cart (`CartController: GetCart, AddToCart, UpdateQuantity, RemoveFromCart, ClearCart`).
- An authenticated user shall be able to add and remove wishlist items and list their wishlist (`WishlistController`).
- An authenticated user shall be able to complete checkout with shipping details, apply a discount, and be redirected to a Paymob payment session (`CartController.CheckOut`).
- The system shall process Paymob's server-to-server callback and the customer-facing payment result page, updating the local order and payment records accordingly (`PaymobCallback, PaymobResult`).

Reviews and Discounts

- Any visitor shall be able to read reviews and rating summaries for an item, and batch rating summaries for multiple items (`ReviewController.GetByItem, GetRatingSummary, GetBatchSummaries`).
- An authenticated user shall be able to create, update, and delete their own review for an item.
- An authenticated user shall be able to validate a promotion code or automatic discount against their current cart total (`DiscountController.Apply`).

3.5 Admin Features

- An administrator shall manage brands, categories, items, and item variants through dedicated CRUD controllers (`BrandController, CategoriesController, ItemsController, ItemVariantsController`), including category/brand selection data for item forms and photo URL handling for variants.
- An administrator shall manage discount rules (create, update, delete, list, get by id) through `DiscountController`.
- An administrator shall review and filter orders by search text, status, and date range, and update order status (`AdminController.Sales, UpdateOrderStatus`).
- An administrator shall view a dashboard summarizing catalog and sales data, with category/brand filters (`AdminController.Index`).
- An administrator shall broadcast a notification to all users (`NotificationController.BroadcastToAll`).

3.6 Notifications & Reporting

- The system shall deliver notifications to users in real time through a SignalR hub at /hubs/notifications and persist them for later retrieval.
- An authenticated user shall be able to list their notifications (optionally unread only), get an unread count, mark one notification as read, and mark all as read (NotificationController).
- The system shall provide a dashboard reporting endpoint that filters non-deleted orders by date, category, and brand, returning KPI values and chart datasets as JSON (DashboardController.GetDashboardData).

4. Non-Functional Requirements

4.1 Performance

- Read-only queries use `AsNoTracking()` to avoid unnecessary EF Core change-tracking overhead.
- List and summary screens use projection (Select) to reduce the amount of data materialized and transferred.
- Related data is loaded deliberately through projection or Include to avoid N+1 query patterns.
- Frequently accessed, low-volatility data is cached using `MemoryCache` at the BLL layer.
- I/O-bound operations (database, file, email, external API calls) use `async/await` throughout, and code must not block asynchronous work with `.Result`, `.Wait()`, or `.GetAwaiter().GetResult()`.

4.2 Security

- Authentication and password storage are handled by ASP.NET Core Identity; the application does not implement custom credential storage.
- State-changing MVC form actions require a valid antiforgery token (`[ValidateAntiForgeryToken]`).
- All database access goes through Entity Framework Core with parameterized queries; user input is never concatenated into SQL.
- Connection strings, SMTP passwords, Google OAuth credentials, and Paymob credentials are kept out of source control and configured through user secrets or deployment environment variables.
- API keys, HMAC secrets, OAuth client secrets, SMTP passwords, and complete payment payloads are never written to logs.
- Paymob callback and payment status data are validated before an order is updated, and the callback path is treated as a sensitive, provider-specific integration point.
- The application is served over HTTPS in both local and deployed environments, with the development certificate trusted locally.
- Exception details and stack traces are not exposed to end users; unhandled exceptions in non-development environments are routed to a generic error page.

4.3 Scalability

- The strict PL → BLL → DAL dependency direction and scoped service lifetimes keep request handling stateless per request, which supports horizontal scaling of the web tier.
- Repository and unit-of-work abstractions isolate persistence concerns, allowing data-access optimizations (e.g., indexing, read replicas) without changing calling code.
- `MemoryCache` reduces repeated load on the database for reference/catalog data that changes infrequently.
- `SignalR` notifications currently target a single application instance; scaling the notification hub across multiple instances would require a backplane (e.g., Redis) as a future enhancement.

4.4 Availability

- Non-development environments use `ExceptionHandler("/Error/500")` so unhandled exceptions surface a controlled error page rather than a raw exception.
- `UseStatusCodePagesWithReExecute` routes HTTP status codes (400, 401, 403, 404, 405, 408, 409, 422, 429, 500, 502, 503, 504) through `ErrorController` to dedicated, user-friendly views.
- Database migrations are designed to be inspected before being applied, reducing the risk of destructive schema changes causing downtime.
- Operational guidance calls for backing up shared databases before schema changes and using separate databases/credentials per environment (development, testing, staging, production).

4.5 Usability

- The storefront provides consistent filtering, sorting, and search-suggestion affordances across both catalog entry points (Home and Store controllers).
- Static informational pages (About, Help, Terms, Contact, Privacy) support customer self-service and transparency.
- Form validation is performed both client-side (model binding/validation attributes) and server-side (model state checks) before any state change.
- Notification and cart/wishlist counts are available through lightweight API endpoints suited to real-time UI badges and updates.
- Administrative screens follow a consistent create/edit/delete-and-redirect pattern with `TempData` success/error messaging, keeping the admin experience predictable across catalog entities.

5. System Architecture

5.1 Architecture Overview

Mazeed+ follows a three-layer architecture within a single ASP.NET Core MVC solution: Mazeed.PL (presentation), Mazeed.BLL (business logic), and Mazeed.DAL (data access). The dependency direction is strictly Mazeed.PL → Mazeed.BLL → Mazeed.DAL → SQL Server. Controllers never query the database directly; they call service abstractions, which in turn use repositories and a unit of work over ApplicationDbContext.

5.2 System Components

Component	Responsibility
Mazeed . PL	MVC presentation: controllers, Razor views, view models binding, static assets, and Program.cs (middleware pipeline, routing, SignalR hub mapping).
Mazeed . BLL	Application services, service abstractions/implementations, AutoMapper mappings, helpers, SignalR hub logic, and email templates.
Mazeed . DAL	Entities, ApplicationDbContext, repositories, seeders, and EF Core migrations.
SQL Server	System of record for Identity, catalog, ordering, and notification data.
Paymob	External payment gateway used for checkout payment sessions and callbacks.
SMTP provider	External email delivery for confirmation, password reset, and contact messages.
Google OAuth	External identity provider for social sign-in.

5.3 Application Architecture

Program.cs configures MVC, application and infrastructure services, MemoryCache, authentication cookies, HTTPS, static files, routing, and the SignalR hub at /hubs/notifications. Dependency injection is centralized through the extension methods AddApplicationServices and AddInfrastructureServices, keeping service registration out of individual controllers. Services and repositories are registered with scoped lifetime to match ApplicationDbContext; stateless integrations such as email are registered as transient. Scoped services (including ApplicationDbContext and repositories) are never injected into singleton components.

Controllers follow a consistent workflow for state-changing actions: bind to a view model, validate model state, check authorization/ownership, call exactly one BLL service operation, translate the service response into a view/redirect/API status code, and return a safe user-facing message without exposing internal exceptions.

5.4 Data Flow

A typical request moves through the layers as follows: the controller validates input and authorization, delegates to a BLL service, the service applies business rules and maps between entities and view models via AutoMapper, and the DAL executes the operation against AppDbContext and SQL Server through EF Core. The result flows back up the same path to produce a Razor view, redirect, or JSON API response. Real-time events (such as new notifications) are pushed out-of-band to connected clients via NotificationHub and SignalRNotificationPusher, independent of the originating request/response cycle.

5.5 External Integrations

Integration	Purpose	Where it is used
Paymob	Payment session creation, payment status callback, and result confirmation.	CartController: CheckOut, PaymobCallback, PaymobResult.
SMTP (EmailSettings)	Registration confirmation, password reset, and contact form messages.	AuthController and HomeController.Contact via IEmailService.
Google OAuth (Authentication:Google)	External sign-in and first-time external registration.	AuthController: ExternalLogin, ExternalLoginCallback, ExternalLoginConfirmation.
SignalR (/hubs/notifications)	Real-time delivery of notifications to connected clients.	NotificationHub, SignalRNotificationPusher, NotificationController.

6. Database Design

6.1 Database Overview

Mazeed+ uses SQL Server with Entity Framework Core 8 and a code-first model in `Mazeed.DAL`. `AppDbContext` inherits from `IdentityDbContext<User, IdentityRole<long>, long>`, combining ASP.NET Core Identity tables with the e-commerce domain model in a single database. All domain entities use long identity keys and share a common audit/soft-delete shape defined by `BaseEntity`:

`Id, CreatedAt, CreatedBy, UpdatedAt, UpdatedBy, IsDeleted, DeletedAt, DeletedBy`

`BaseEntity` and `IBaseEntity` are shared abstractions only; they do not create standalone tables. `AppDbContext` explicitly maps Identity tables to `Users`, `Roles`, `UserRoles`, `UserClaims`, `RoleClaims`, `UserLogins`, and `UserTokens`, and defines composite keys for the cart, favorites, notifications, and item-category join relationships.

6.2 Entity Relationship Diagram (ERD)

The physical ERD below reflects the DbSet declarations and relationships configured in `AppDbContext.OnModelCreating`. The authoritative relationship definitions live in that method; this document and the Draw.io ERD at `docs/ERD.drawio` are kept consistent with it.

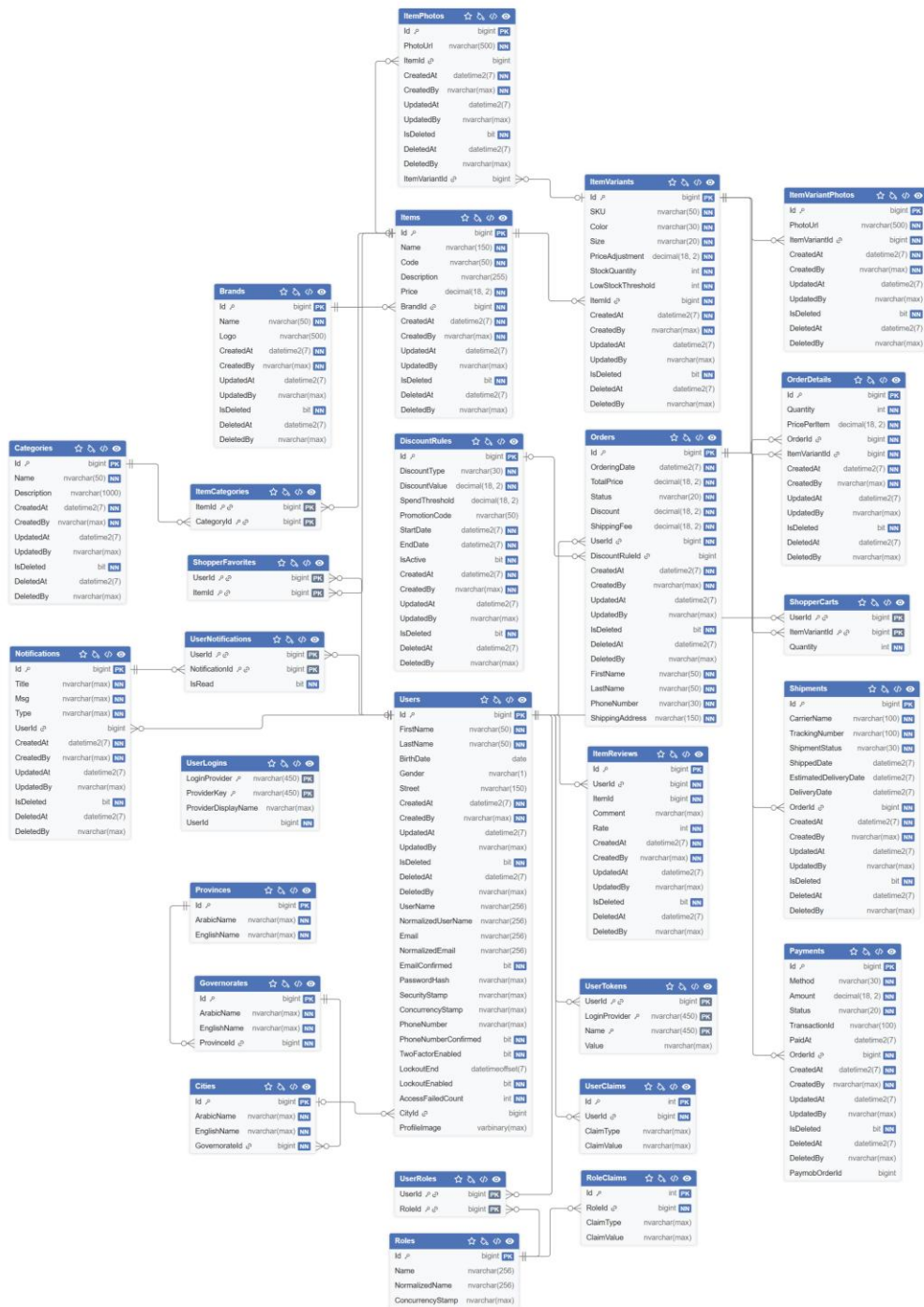


Figure 6.1 — Mazeed+ physical entity relationship diagram (source: `physical_erd.png`).

6.3 Entities & Attributes

The table below lists every entity mapped by ApplicationDbContext, its corresponding database table, its category, and its purpose. Unless noted otherwise, domain entities inherit from BaseEntity and use a long identity key.

Entity	Table	Type	Purpose
User	Users	Identity entity	Application user profile and ASP.NET Core Identity account data.
Province	Provinces	Domain entity	Top-level geographical location.
Governorate	Governorates	Domain entity	Governorate linked to a province.
City	Cities	Domain entity	City linked to a governorate.
Brand	Brands	Domain entity	Product brand and logo information.
Category	Categories	Domain entity	Product classification and descriptions.
Item	Items	Domain entity	Main product record, price, code, and brand relationship.
ItemPhoto	ItemPhotos	Domain entity	Product-level image references.
ItemVariant	ItemVariants	Domain entity	Sellable variation (size, color, SKU, price, stock).
ItemVariantPhoto	ItemVariantPhotos	Domain entity	Image references for a specific variant.
ItemCategory	ItemCategories	Join entity	Many-to-many between items and categories. Composite key: ItemId, CategoryId.
Order	Orders	Domain entity	Customer order, totals, status, shipping, and payment references.
OrderDetail	OrderDetails	Domain entity	Ordered variant, quantity, and captured item price.
Payment	Payments	Domain entity	Payment provider, method, status, and transaction info for an order.
Shipment	Shipments	Domain entity	Delivery and shipment information for an order.
DiscountRule	DiscountRules	Domain entity	Promotion code or automatic discount definition and validity period.
ItemReview	ItemReviews	Domain entity	Shopper rating and review for an item.
ShopperCart	ShopperCarts	Join entity	Selected item variants and quantities. Composite key: UserId, ItemVariantId.
ShopperFavorite	ShopperFavorites	Join entity	Wishlist relationship. Composite key: UserId, ItemId.
Notification	Notifications	Domain entity	Notification content and metadata.
UserNotification	UserNotifications	Join entity	Delivery/read state. Composite key: UserId, NotificationId.

ASP.NET Core Identity Tables

Identity type	Table	Purpose
IdentityRole<long>	Roles	Application roles such as Admin.
IdentityUserRole<long>	UserRoles	User-to-role assignments.
IdentityUserClaim<long>	UserClaims	Claims assigned to users.
IdentityRoleClaim<long>	RoleClaims	Claims assigned to roles.
IdentityUserLogin<long>	UserLogins	External login provider associations.
IdentityUserToken<long>	UserTokens	Authentication and provider tokens.

6.4 Primary & Foreign Keys

Every domain entity uses a long, database-generated Id as its primary key, except for join entities, which use composite primary keys formed from their two foreign keys:

- ItemCategories — composite key (ItemId, CategoryId), foreign keys to Items and Categories.
- ShopperCarts — composite key (UserId, ItemVariantId), foreign keys to Users and ItemVariants, with an additional Quantity column.
- ShopperFavorites — composite key (UserId, ItemId), foreign keys to Users and Items.
- UserNotifications — composite key (UserId, NotificationId), foreign keys to Users and Notifications, with an additional IsRead column.

Notable single-column foreign keys include: Items.BrandId → Brands.Id; ItemVariants.ItemId → Items.Id; ItemPhotos.ItemVariantId → ItemVariants.Id; ItemVariantPhotos.ItemVariantId → ItemVariants.Id; Orders.UserId → Users.Id and Orders.DiscountRuleId → DiscountRules.Id; OrderDetails.OrderId → Orders.Id and OrderDetails.ItemVariantId → ItemVariants.Id; Payments.OrderId → Orders.Id; Shipments.OrderId → Orders.Id; ItemReviews.UserId → Users.Id and ItemReviews.ItemId → Items.Id; Cities.GovernorateId → Governorates.Id; Governorates.ProvinceId → Provinces.Id; and Users.CityId → Cities.Id.

6.5 Relationships

- A brand owns many items; an item can belong to many categories through ItemCategories.
- An item owns item variants and item photos; variants can have their own photos.
- An order belongs to a shopper (user) and contains many order details. Each detail references an item variant.
- An order can have one payment and one shipment.
- A discount rule can be associated with many orders.
- A shopper has cart and favorite (wishlist) relationships, both represented by composite keys.
- Notifications are linked to users through UserNotifications, capturing per-user read state.
- Item reviews reference both the reviewed item and the shopper who submitted the review.
- Geography is modeled hierarchically: Province → Governorate → City, with Users referencing a City.

6.6 Database Constraints

- Table names use plural PascalCase and Id primary keys.
- New domain entities use the existing long key type unless a relationship requires otherwise.
- Money values (prices, payments, orders, discounts) use decimal(18,2).
- Timestamps are stored in UTC, consistent with BaseEntity.CreatedAt.
- Foreign keys and navigation properties are configured centrally in AppDbContext or the established entity configuration pattern.
- Soft-delete fields follow BaseEntity; a global query filter is never assumed without being explicitly added and reviewed in AppDbContext.
- Every required string has an explicit maximum length and nullability rule in entity configuration or data annotations.
- Explicit delete behavior is defined for relationships where deleting a parent could affect orders, reviews, payments, or other historical records.
- Payment secrets or raw credentials are never stored in entities, seed data, migrations, or logs.

7. System Features

7.1 Feature Overview

This chapter describes Mazeed+'s user-facing features in detail: the storefront discovery experience, shopping and checkout, reviews, notifications, and the administrative back office. Each feature is described with its specification, the screens involved, its workflow, and the business rules that govern it.

7.2 Feature Specifications

Feature	Specification
Catalog browsing	Two public entry points (HomeController and StoreController) support filtering by category, brand, price, color, and size, free-text search, and sorting, with a recommendation-based default order when no explicit sort is chosen.
Item details	Displays an item with its variants, available colors/sizes, and related or frequently-bought-together items.
Cart	Authenticated users add, update quantity, remove, view, and clear cart lines, keyed by item variant.
Wishlist	Authenticated users add, remove, and list favorite items, keyed by item.
Checkout & payment	Shipping details are captured, a discount may be applied, an order is created, and a Paymob payment session is started; Paymob's callback and result pages complete the payment/order lifecycle.
Reviews & ratings	Any visitor can read reviews and rating summaries (including in batch); authenticated users manage their own review per item.
Discounts	Administrators manage promotion codes and automatic spend-threshold discount rules; shoppers apply a code or automatic discount at checkout.
Notifications	Users receive real-time notifications through SignalR, can view unread counts, mark items read, and administrators can broadcast to all users.
Admin catalog management	Brand, category, item, and item variant CRUD screens restricted to the Admin role, each following the same create/edit/delete-and-redirect pattern.
Admin sales & dashboard	Order search/filter and status update screen, plus a KPI/chart dashboard filterable by date, category, and brand.
Admin user & role management	User CRUD, role CRUD, and role-assignment management restricted to the Admin role.

7.3 User Interface Screens

The table below summarizes the primary screens/routes exposed by the application. Live, current screenshots can be viewed directly on the deployed application; see Appendix C.

Screen	Route	Audience
Home / catalog	/ , /Home/Index, /Store/Index	Public
Category listing	/Home/Category	Public
Item details	/Home/Details/{id}	Public
About / Help / Terms / Privacy / Contact	/Home/About, /Home/Help, /Home/Terms, /Home/Privacy, /Home/Contact	Public
Register / Login / Forgot / Reset Password	/Auth/Register, /Auth/Login, /Auth/ForgotPassword, /Auth/ResetPassword	Anonymous
Profile	/Auth/Profile	Authenticated
Cart / Checkout	/Cart, /Checkout	Authenticated
Payment result	/Checkout/PaymobResult	Authenticated (provider return)
Notifications	/Notifications	Authenticated
Admin dashboard	/Admin/Index, /Dashboard/Index	Admin
Admin sales	/Admin/Sales	Admin
Admin users	/Admin/Users	Admin
Catalog management	/Brand, /Categories, /Items, /ItemVariants (Index/Create/Edit/Delete)	Admin
Error pages	/Error/{statusCode}	Public (error handling)

7.4 Feature Workflows

Checkout Workflow

- The shopper opens /Cart and reviews cart contents, then proceeds to /Checkout.
- The shopper submits shipping details (name, phone, address) and optionally a discount code.
- CartController validates the model, creates the order and order details from the current cart, applies any valid discount, and starts a Paymob payment session.
- The shopper is redirected to the Paymob-hosted payment page to complete payment.
- Paymob calls back to /Checkout/PaymobCallback (server-to-server) with the payment status; the application validates the payload and completes the local payment record.
- The shopper is returned to /Checkout/PaymobResult, which resolves the order, confirms the recorded payment status, and displays the outcome.

Add-to-Cart Workflow

- The shopper selects an item variant (color/size) on the item details page.
- The client calls POST /api/cart with the selected variant and quantity.
- CartController.AddToCart validates the request and calls the cart service, which persists or updates the ShopperCart's composite-key row.
- The updated cart state (and count) is returned to the client for UI refresh.

Admin Catalog Change Workflow

- An administrator opens the relevant management screen (e.g., /Items).
- The administrator submits a Create/Edit/Delete form, protected by an antiforgery token.
- The controller validates model state, calls the corresponding BLL service, sets a TempData success or error message, and redirects back to Index.

7.5 Business Rules

- State-changing MVC form actions require a valid antiforgery token, except the Paymob server-to-server callback, which intentionally bypasses antiforgery because it is not a browser-originated request.
- Admin-only controllers and actions enforce the Admin role at the class or action level; unauthenticated API calls receive 401 rather than a login redirect.
- A discount code or automatic discount is validated against the current cart total before being applied at checkout.
- Payment callback processing must be idempotent — a duplicate callback for the same transaction must not double-apply a payment or change order state incorrectly.
- Deleting a parent record that has historical orders, reviews, or payments must use an explicit delete behavior rather than an unreviewed cascade.
- Soft-deleted records (IsDeleted = true) are excluded from active dashboard/reporting queries, which explicitly filter for non-deleted orders.

8. Deployment & Installation

8.1 System Requirements

- .NET 8 SDK
- SQL Server (LocalDB, Express, Developer, or another reachable SQL Server instance)
- Visual Studio with the ASP.NET and web development workload, or VS Code with C# tooling
- Git
- Paymob, SMTP, and Google OAuth credentials for the related features

The SDK can be verified from a terminal with: `dotnet --version`

8.2 Installation Guide

Repository Structure

The solution file is located under `src`, so commands that reference projects should be run from that directory unless the command explicitly uses a path from the repository root:

- `repository-root/src/Mazeed.sln`, `Mazeed.PL/`, `Mazeed.BLL/`, `Mazeed.DAL/`
- `repository-root/docs/ERD.drawio`
- `repository-root/wiki/`

Restore and Build

From the repository root:

```
cd src
dotnet restore Mazeed.sln
dotnet build Mazeed.sln
```

To enable local HTTPS, trust the development certificate when needed: `dotnet dev-certs https --trust`

Database Initialization

Start SQL Server, then apply the checked-in EF Core migrations from the `src` directory:

```
dotnet ef database update --project Mazeed.DAL --startup-project Mazeed.PL
```

The database model is defined in `Mazeed.DAL/Database/AppDbContext.cs`. Seed data is registered through `DbInitializer` and the seeders in `Mazeed.DAL/Seeders` when migrations are generated/applied; no `--seed` command-line flag is required. The initial seed set includes roles, an administrator, geographical data, categories, brands, items, variants, and photos. Seeded credentials must be handled as sensitive configuration and changed according to the team's deployment process.

Run the Application

From the `src` directory: `dotnet run --project Mazeed.PL` — or open `src/Mazeed.sln` in Visual Studio and run `Mazeed.PL`. The terminal or IDE displays the actual HTTP and HTTPS URLs for the current launch profile.

First-Run Verification

- Open the printed HTTPS URL and confirm the home page loads.
- Browse a category and open an item with variants.
- Confirm registration/login and cookie authentication work with local data.
- If configured, test one email, Google sign-in, and Paymob flow in a non-production environment.
- Confirm a notification-producing action reaches the user through the application and SignalR connection.

8.3 Configuration

Mazeed.PL reads the DefaultConnection connection string and the following configuration sections: EmailSettings, Authentication:Google, and Paymob. Real passwords, tokens, OAuth secrets, and payment credentials must never be committed; configure local values with user secrets:

```
cd src/Mazeed.PL
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:DefaultConnection"
"Server=.;Database=MazeedDb;Trusted_Connection=True;TrustServerCertificate=True;"
dotnet user-secrets set "Authentication:Google:ClientId" "your-client-id"
dotnet user-secrets set "Authentication:Google:ClientSecret" "your-client-secret"
dotnet user-secrets set "Paymob:ApiKey" "your-paymob-api-key"
dotnet user-secrets set "EmailSettings:Password" "your-smtp-password"
```

Add the remaining Paymob values (IntegrationId, IFrameId, HmacSecret, BaseUrl) and email values (Host, Port, SenderEmail, SenderName, Username, EnableSsl, AdminEmail) through the same mechanism. User secrets override values in appsettings.json and appsettings.Development.json. For a local machine, the minimum useful configuration is a SQL Server connection string; email, Google sign-in, and Paymob only require valid credentials when those flows are being tested. Production credentials must never be copied into a local configuration file, and any credential that has been committed or shared must be revoked and rotated.

The application uses cookie authentication with login path /Auth/Login, logout path /Auth/Logout, and access-denied path /Error/403. API requests that are unauthenticated receive 401 rather than an HTML login redirect.

8.4 Deployment Process

The application is currently deployed and reachable at the live demo URL: <https://mazeed.runasp.net/>. A recommended deployment process, consistent with the project's migration and configuration practices, is:

- Build and publish Mazeed.PL for the target environment.
- Provision a SQL Server database and configure connection, SMTP, Google OAuth, and Paymob credentials as deployment environment variables or a secret store — never in source control.
- Generate and review an idempotent migration script before applying it to a shared database: dotnet ef migrations script --idempotent --project Mazeed.DAL --startup-project Mazeed.PL.
- Apply migrations to the target database, back up shared databases beforehand, and confirm Identity and catalog tables exist before enabling traffic.
- Deploy the published application behind HTTPS, and verify the smoke-check steps in Section 10.2 against the deployed URL.
- Confirm the SignalR hub endpoint (/hubs/notifications) is reachable through the deployment's networking/reverse-proxy configuration.

8.5 Environment Variables

Configuration key	Purpose
ConnectionStrings:DefaultConnection	SQL Server connection string for AppDbContext.
Authentication:Google:ClientId / ClientSecret	Google OAuth credentials for external sign-in.
Paymob:ApiKey / IntegrationId / IframeId / HmacSecret / BaseUrl	Paymob payment gateway configuration.
EmailSettings:Host / Port / SenderEmail / SenderName / Username / Password / EnableSsl / AdminEmail	SMTP configuration for confirmation, reset, and contact emails.

9. Maintenance & Support

9.1 Maintenance Guidelines

Migration Safety Checklist

- Confirm the entity and relationship change is intentional.
- Update or add seed data only when it belongs to the schema change.
- Run the build from src and verify the correct startup project.
- Create the migration with a descriptive PascalCase name, such as AddShipmentTrackingFields.
- Inspect the generated operations for unintended table drops, column renames, nullability changes, or data loss.
- Apply the migration to a disposable or development database before a shared environment.
- Update docs/ERD.drawio and the database guide if the domain model changed.

Never edit an applied migration to correct a later change; create a new migration instead. Remove a migration only when it has not been applied to a shared database.

9.2 Troubleshooting

Problem	Check
Cannot open the database	Confirm SQL Server is running and ConnectionStrings:DefaultConnection points to the intended server/database.
Login or certificate error	Add TrustServerCertificate=True for local SQL Server and verify the database credentials.
Missing Paymob, Google, or SMTP configuration	Check the exact section/key names in Section 10.5 and confirm user secrets are configured for Mazeed.PL.
HTTPS warning	Run dotnet dev-certs https --trust, then restart the browser.
Application startup failure	Run dotnet run --project Mazeed.PL from src to see the complete exception.
EF command cannot find a project	Run the command from src, or provide paths relative to the repository root such as --project src/Mazeed.DAL --startup-project src/Mazeed.PL.
Changes are not visible	Stop and restart the application, confirm the expected environment is loaded, and check that the browser is using the URL printed by the current launch profile.

9.3 Future Improvements

- Introduce an automated test project (unit and integration) using Arrange-Act-Assert, targeting services, controllers, and payment/authentication flows.
- Review and, if intended to be administrator-only, explicitly authorize `DashboardController`, which currently declares no authorization attribute in the source despite exposing sales and customer analytics.
- Add a SignalR backplane (e.g., Redis) if the notification hub needs to scale across multiple application instances.
- Expand automated CI checks (build, and eventually test execution) alongside the existing manual review checklist.
- Consider multi-language storefront support and additional payment or shipping providers as the catalog grows.
- Formalize the test-result log introduced in Section 9.6 as part of the release process.

10. Conclusion

10.1 Project Summary

Mazeed+ delivers a complete, layered ASP.NET Core MVC e-commerce application: catalog discovery, cart and wishlist, checkout with Paymob payments, reviews, real-time notifications via SignalR, and a full administrative back office, all built on Entity Framework Core 8 and SQL Server with ASP.NET Core Identity for authentication. The project demonstrates the practical application of layered architecture, secure authentication, external payment integration, and administrative reporting within the ITI Summer Training 2026 Full Stack .NET track.

10.2 Achievements

- A working, deployed application reachable at <https://mazeed.runasp.net/>.
- A clean PL → BLL → DAL layered architecture with consistent controller, service, and repository patterns.
- A rich, normalized database model covering catalog, ordering, payments, shipments, discounts, reviews, and notifications, integrated with ASP.NET Core Identity.
- Real third-party integrations: Paymob for payments, Google OAuth for sign-in, SMTP for transactional email, and SignalR for real-time notifications.
- A functioning administrative back office covering catalog CRUD, order management, discount rules, user/role management, and dashboard analytics.
- Documented coding, database, and review standards that keep the codebase consistent across contributors.

10.3 Limitations

- No automated test project exists yet; quality currently relies on the manual verification checklist in Section 9.1.
- `DashboardController` does not declare an authorization attribute in the current source, which should be reviewed before treating dashboard data as administrator-only in all deployments.
- The notification hub is designed for a single application instance; multi-instance scaling would require additional infrastructure (a SignalR backplane).
- The storefront currently targets a single language and currency.
- As with any project relying on third-party providers (Paymob, Google, SMTP), functionality that depends on those integrations requires valid, provider-specific credentials to fully verify in any given environment.

10.4 Future Enhancements

- Add unit and integration test coverage across services, controllers, and payment/authentication flows.
- Close the `DashboardController` authorization gap and add automated authorization tests for all admin-only surfaces.
- Introduce a SignalR backplane for horizontal scaling of real-time notifications.
- Expand reporting/analytics on the admin dashboard (e.g., cohort or repeat-purchase metrics).
- Consider additional payment providers, shipping carriers, and localization (language/currency) as the product matures.

Appendices

A. Database Schema

The complete entity/table inventory and Identity table mappings are provided in Section 6.3. The authoritative schema is defined in `Mazeed.DAL/Database/AppDbContext.cs` and its EF Core migrations under `Mazeed.DAL/Migrations`; the physical ERD is reproduced in Section 6.2 and maintained at `docs/ERD.drawio`.

B. API Reference

A consolidated reference of every controller in `Mazeed.PL/Controllers`, its primary responsibility, and its access level:

Controller	Primary responsibility	Access
HomeController	Public home page, catalog filtering, item details, static pages, and contact form.	Public
StoreController	Public catalog search, filtering, sorting, recommendations, and autocomplete.	Public
AuthController	Registration, login, email confirmation, password reset, Google sign-in, profile, logout.	Mixed public/authenticated
CartController	Authenticated cart, checkout, Paymob payment initiation, callback, and result handling.	Authenticated, except Paymob callback/result
WishlistController	Authenticated wishlist operations.	Authenticated
ReviewController	Public review reads and authenticated review changes.	Mixed public/authenticated
NotificationController	Authenticated notification reads/updates, plus admin broadcasts.	Authenticated; broadcast is admin-only
DiscountController	Admin discount-rule management and authenticated promo-code application.	Mixed admin/authenticated
BrandController	Admin brand CRUD.	Admin role
CategoriesController	Admin category CRUD.	Admin role
ItemsController	Admin item CRUD and category/brand selection data.	Admin role
ItemVariantsController	Admin variant CRUD and photo URL handling.	Admin role
AdminController	Admin dashboard, users, roles, sales, and order status management.	Admin role
DashboardController	Dashboard view and filtered sales analytics JSON.	See Section 12.3 authorization note
ErrorController	Status-code-specific error pages.	Public error handling
ApiBaseController	Shared API response and current-user helpers.	Abstract base class

Detailed, per-action routes for the public storefront, authentication, shopping/customer APIs, notifications, discounts, and administration controllers are provided in Sections 3, 7.3, 7.4, and 8.3 of this document.

C. Screenshots

A physical entity relationship diagram is included as Figure 6.1 in Section 6.2. Live, up-to-date screenshots of every screen are best captured directly from the running application, since the UI evolves independently of this document. The deployed application is available at: <https://mazed.runasp.net/>

Recommended screens to capture for a project presentation or report: the home catalog page, an item details page with variants, the cart and checkout flow, the profile/order-history page, the notification center, and the admin dashboard, sales, and catalog-management screens.

D. Glossary

Term	Definition
MVC	Model-View-Controller — the architectural pattern used by the ASP.NET Core presentation layer.
EF Core	Entity Framework Core — the object-relational mapper used for data access against SQL Server.
Code-first	An approach where the database schema is generated from C# entity classes rather than designed first in the database.
Identity	ASP.NET Core Identity — the framework providing user, role, claim, login, and token management for authentication.
BLL	Business Logic Layer (Mazed.BLL) — hosts services, mappings, and business rules.
DAL	Data Access Layer (Mazed.DAL) — hosts entities, AppDbContext, repositories, seeders, and migrations.
Unit of Work	A pattern that coordinates one or more repositories within a single transaction/save operation.
Repository	An abstraction that encapsulates persistence operations for an entity or aggregate.
ViewModel	A presentation-layer contract used to bind and validate request/response data, distinct from a DAL entity.
ServiceResponse	A standard result shape returned by BLL services, indicating success/failure and carrying data or messages.
SKU	Stock Keeping Unit — a unique identifier for a specific sellable item variant.
SignalR	A .NET library that enables real-time, bidirectional communication between server and client, used here for notifications.
Paymob	The third-party payment gateway integrated for Mazed+ checkout.
Soft delete	Marking a record as deleted (IsDeleted, DeletedAt, DeletedBy) instead of physically removing it from the database.

Term	Definition
Migration	An EF Core-generated, versioned set of schema changes applied to keep the database in sync with the entity model.
ERD	Entity Relationship Diagram — a visual representation of entities and their relationships.

E. References

- Home.md — Mazeed+ Wiki home and feature overview.
- Setup.md — Local setup, configuration, and troubleshooting guide.
- Standards.md — Coding standards: layering, naming, services, controllers, security, and review expectations.
- Database.md — Database guide: entities, Identity tables, EF Core migrations, seed data, and conventions.
- Controllers.md — Controller reference: MVC controllers, API routes, authorization, and action responsibilities.
- physical_erd.png — Physical entity relationship diagram (reproduced as Figure 6.1).
- Live demo: <https://mazeed.runasp.net/>